

Document-level Relation Extraction & Knowledge Graph 시스템

최종 보고서

DocRED 기반 End-to-End 정보 추출 파이프라인 설계 및 구현

ATLOP + DREEAM + GAIN 구조 기반 3단계 Incremental Stacking 설계

통합 시나리오: 이기종 그래프 및 U-Net 구조 도입 분석 포함

작성일: 2026년 4월

목차 (Table of Contents)

1. 프로젝트 개요 및 문제 정의
2. 데이터셋: DocRED
3. 평가 지표 및 기존 논문 성능 비교
4. 3단계 Incremental Stacking 로드맵
5. 전체 레이어 상세 요약 표
6. Stage 1 — Baseline RE 모델
7. Stage 2 — ATLOP + DREEAM 강화
8. Stage 3 — Graph Reasoning (GAIN-lite)
9. 통합 시나리오: 계층별 아키텍처 변화 분석
10. Knowledge Graph Layer (Neo4j)
11. 구현 계획 및 코드 구조
12. 팀원 역할 분담
13. 핵심 참고 논문

1. 프로젝트 개요 및 문제 정의

본 프로젝트는 **DocRED 데이터셋**을 기반으로 문서 수준 관계 추출(Document-level Relation Extraction)을 수행하고, 추출된 관계를 Knowledge Graph로 확장하여 Graph-based Reasoning 시스템으로 발전시키는 것을 목표로 한다. 핵심은 **단순 RE 모델이 아닌 End-to-End 파이프라인**을 구축하고, 3단계 Incremental Stacking 방식으로 모델을 확장하여 각 모듈의 효과를 정량적으로 검증하는 데 있다.

1.1 Document-level RE 란?

문서 수준 관계 추출(DocRE)은 하나의 문서 내에 등장하는 entity 쌍에 대해 어떤 관계가 존재하는지 예측하는 태스크이다. 기존 문장 수준 RE와의 핵심 차이는 다음과 같다.

항목	Sentence-level RE	Document-level RE (본 프로젝트)
입력 범위	단일 문장	전체 문서 (multi-sentence)
관계 단서	문장 내 로컬 정보	문서 전체 + inter-sentence 단서
출력 형식	단일 라벨 분류	Multi-label (복수 관계 허용)
추론 방식	직접 언급된 관계만	Multi-hop 간접 추론 포함

1.2 핵심 어려움

Cross-sentence reasoning: 하나의 관계를 확인하는 단서가 여러 문장에 분산

Multi-hop inference: $A \rightarrow B$, $B \rightarrow C$ 정보로부터 $A \rightarrow C$ 관계를 간접 추론

Multi-label: 하나의 entity 쌍이 동시에 여러 관계를 가질 수 있음

클래스 불균형: 실제 관계(positive)에 비해 no_relation pair가 압도적으로 많음

Long-context: DocRED 문서는 평균 약 200 토큰 이상, 일부는 512 초과

1.3 전체 파이프라인 요약

단계	내용	상세
① Preprocessing	DocRED → Feature 변환	entity mention alignment, pair 생성
② Document Encoder	BERT-base 인코딩	Contextual token representation
③ Entity Representation	Mention → Entity 벡터	LogSumExp pooling
④ Structural Reasoning	전역 구조 추론 (통합 시나리오)	이기종 그래프 / U-Net (Stage 3)
⑤ Relation Extraction	Multi-label Classification	Adaptive Threshold (ATLOP)
⑥ Evidence Head	DREEAM evidence-guided attention	Stage 2에서 추가
⑦ Postprocessing	Triple 정제 및 필터링	Dedup, normalization
⑧ Knowledge Graph	Neo4j 그래프 구성	Node/Edge 저장

2. 데이터셋: DocRED

DocRED (Document-level Relation Extraction Dataset)는 Yao et al. (2019, ACL)이 발표한 문서 수준 관계 추출을 위한 대규모 벤치마크 데이터셋이다. Wikipedia 문서에서 구축되었으며, Wikidata의 관계 정보를 기반으로 annotation되었다.

2.1 데이터셋 통계

항목	값
문서 수 (Human-annotated)	5,053 (train 3,053 / dev 1,000 / test 1,000)
문서 수 (Distantly supervised)	101,873
Relation 종류	96개 + 1 (no_relation)
Entity type	6개 (PER, LOC, ORG, TIME, NUM, MISC)
평균 문장 수 / 문서	약 8문장
평균 entity 수 / 문서	약 19.5개
평균 mention 수 / entity	약 1.4개
평균 relation triple 수 / 문서	약 12.5개
Inter-sentence relation 비율	약 40.7% (문서 수준 RE의 핵심)

2.2 데이터 구조 (JSON)

각 문서는 다음 필드로 구성된다:

```
{
  "title": "...",
  "sents": [{"token1", "token2", ...}, ...], // 문장별 토큰 리스트
  "vertexSet": [                                // entity 목록
    [{"name": "...", "sent_id": 0, "pos": [2, 4], "type": "PER"}, ...]
  ],
  "labels": [                                    // relation 정답
    {"h": 0, "t": 1, "r": "P17", "evidence": [0, 2]}
  ]
}
```

2.3 학습 전략: Distantly Supervised Pre-training

본 프로젝트에서는 DREEAM (Ma et al., 2023) 및 KD-DocRE (Tan et al., 2022)의 학습 전략을 참고하여, **distantly supervised data로 pre-training 후 human-annotated data로 fine-tuning**하는 2단계 학습을 기본으로 채택한다.

3. 평가 지표 및 기존 논문 성능 비교

3.1 핵심 평가 지표

지표	설명	적용 단계
Micro F1	모든 relation 예측을 동등 가중치로 평가. DocRE 표준 비교 지표.	전 Stage
Ign F1	훈련/테스트 공유 triple 제외 F1. 더 엄밀한 평가.	전 Stage
Evidence F1	관계의 근거 문장을 얼마나 정확히 찾는지 평가.	Stage 2 (DREEM)
Intra-F1	문장 내 관계만 평가	분석용
Inter-F1	문장 간 관계만 평가 (multi-hop 포함)	분석용 (Stage 3)

3.2 기존 논문 성능 비교 (DocRED Benchmark)

† 표시는 *distantly supervised pre-training* 적용 모델을 의미한다.

BERT-base 기반 모델

모델	Dev Ign F1	Dev F1	Test Ign F1	Test F1	비고
CNN (Yao 2019)	41.58	43.45	40.33	42.26	Baseline
GAIN-BERT† (Zeng 2020)	61.38	63.49	60.60	62.87	Graph-based
SSAN-BERT† (Xu 2021)	61.52	63.55	61.19	63.40	Structure-aware
ATLOP-BERT† (Zhou 2021)	62.23	64.19	61.79	63.90	Adaptive Threshold
KD-DocRE-BERT† (Tan 2022)	—	—	62.31	64.38	Knowledge Distill.
DREEM-BERT† (Ma 2023)	≈ 63.0	≈ 65.0	63.41	65.49	Evidence-aware
GEBA-BERT† (2024)	—	—	63.58	65.93	Graph+Evidence

RoBERTa-large 기반 모델

모델	Dev Ign F1	Dev F1	Test Ign F1	Test F1
SSAN-RoBERTa (Xu 2021)	60.25	62.08	59.47	61.42
ATLOP-RoBERTa (Zhou 2021)	61.32	63.18	61.39	63.40
DocuNet-RoBERTa (Zhang 2021)	62.23	64.12	62.39	64.55
DREEAM-RoBERTa† (Ma 2023)	—	—	≥64	≥66
SAIS-RoBERTa† (Xiao 2022)	62.23	65.17	63.44	65.11

3.3 본 프로젝트 목표 성능

Stage	구성	Dev F1 목표	Test F1 목표	비교 기준
Stage 1	BERT + mean pooling + pairwise classifier	≥ 60%	≥ 59%	ATLOP-BERT† 수준
Stage 2	Stage 1 + ATLOP + DREEAM	≥ 63%	≥ 62%	DREEAM-BERT† 수준
Stage 3	Stage 2 + GNN (GAIN-lite)	Stage 2 +0.5~1.0	Stage 2 대비 향상	Inter-F1 개선 확인

4. 3 단계 Incremental Stacking 로드맵

4.1 핵심 설계 철학

본 프로젝트의 가장 중요한 설계 원칙은 **"Incremental Stacking"**이다. 이전 단계의 코드를 교체하지 않고 그 위에 확장 모듈을 얹는 구조이다:

실험 비교가 완벽함: Stage 1 vs 2 → RE 개선 효과, Stage 2 vs 3 → Graph 효과를 정량적으로 분리 검증 가능

디버깅 용이: 문제 발생 시 Stage 1 → 정상? Stage 2 → 깨짐? 순서대로 추적 가능

코드 수정 최소화: Stage 1 forward 구조를 그대로 유지하면서 모듈만 추가

4.2 Stage 별 구조 요약

Stage 1: Baseline RE

```
Encoder (BERT-base)
  → Entity Representation (mention pooling)
  → Pair-wise Classifier (bilinear + sigmoid)
  → relation_logits
```

Stage 2: Stage 1 + ATLOP + DREEAM

```
Encoder (BERT-base)
  → Entity Representation (LogSumExp pooling)      ← 변경
  → ATLOP Classifier (adaptive threshold)          ← 변경
  → relation_logits
  → evidence_logits                                ← 추가 (DREEAM)
```

Stage 3: Stage 2 + Structural Reasoning (GAIN-lite)

```
Encoder (BERT-base)
  → Entity Representation (LogSumExp pooling)
  → [전역 구조 추론 계층]                            ← 추가
    이기종 GNN: Entity-Sentence-Document 3층 그래프
    또는 U-Net: N×N 2D 쌍 행렬 + CNN
  → Refined Entity Vectors
    → ATLOP Classifier (adaptive threshold)
    → relation_logits + evidence_logits
```

4.3 핵심 차이: Stage 2 vs Stage 3

항목	Stage 2	Stage 3
개선 대상	분류기 (Classifier) 강화	표현 (Representation) 자체 강화
핵심 모듈	ATLOP + DREEAM	이기종 GNN (GAIN) / U-Net (DocuNet)
효과	Adaptive threshold + evidence supervision	문서 전체 entity 간 상호작용 + multi-hop 추론
코드 변경	relation_head 내부 수정	entity_repr → structural_encoder 삽입

5. 전체 레이어 상세 요약 표

아래 표는 본 시스템의 모든 레이어에 대해 **목표**, **사용 모델**, **핵심 알고리즘/기법**, **출력**을 정리한 것이다. 각 레이어가 정확히 어떤 역할을 하고, 어떤 기술을 사용하는지 한눈에 파악할 수 있다.

1. Preprocessing Layer

목표	DocRED raw data를 학습 가능한 입력으로 변환
사용 모델	BERT Tokenizer (bert-base-uncased)
핵심 알고리즘 / 기법	sentence/token 정리, sentence offset mapping, entity mention alignment (sent_id + pos → 문서 전체 인덱스), entity pair generation ($N*(N-1)$ 방향성 쌍), multi-hot label vectorization (96차원)
출력	input_ids, attention_mask, entity_spans, entity_pairs, labels (multi-hot vector)

2. Document Encoder Layer

목표	문서 전체 문맥을 반영한 토큰 표현 생성
사용 모델	BERT-base-uncased (HuggingFace Transformers)
핵심 알고리즘 / 기법	Transformer self-attention ($O(n^2)$), contextual embedding, 최대 512 토큰 입력. 통합 시나리오에서는 문장 경계(Sentence Boundary) 정보를 메타데이터로 추가 추출하여 이기종 그래프 구성에 활용
출력	token-level hidden states $H \in \mathbb{R}^{L \times 768}$

3. Entity Representation Layer

목표	여러 mention을 통합해 entity-level 벡터 생성
사용 모델	Stage 1: Mean Pooling / Stage 2~3: ATLOP 방식 LogSumExp Pooling
핵심 알고리즘 / 기법	mention span pooling ($H[start:end] \rightarrow$ mention vector), mention-to-entity aggregation (동일 entity의 mention들 통합), smooth max approximation ($\text{LogSumExp} = \log \sum \exp$). 통합 시나리오에서도 변경 없이 유지 — 다음 구조적 추론 계층의 입력값으로 전달

출력	entity vectors $e_i \in \mathbb{R}^d$
----	---------------------------------------

4. Structural Reasoning Layer [통합 시나리오 / Stage 3]

목표	문서 전체 entity 간 전역적(Holistic) 상호작용 계산, inter-sentence / multi-hop 관계 추론 강화
사용 모델	GAIN-lite (GCN/GAT 기반 이기종 GNN) 또는 DocuNet (U-Net 기반)
핵심 알고리즘 / 기법	[이기종 GNN 방식] Entity-Sentence-Document 3층 구조 그래프 구성, co-reference edge + co-occurrence edge + cross-sentence edge, 2-hop GCN/GAT message passing. [U-Net 방식] $N \times N$ entity pair 2D 행렬 구성 → CNN 기반 U-Net 레이어로 convolution → (A,B) 쌍 판단 시 (B,C) 인접 픽셀 정보를 압축하여 multi-hop 추론. Stage 1~2의 단순 pair-wise concat $[e_h; e_t; e_h \odot e_t]$ 을 대체하는 핵심 계층
출력	refined entity vectors / refined 2D pair matrix → ATLOP classifier에 전달

5. Relation Extraction Layer

목표	엔티티 쌍 간 관계를 multi-label로 예측
사용 모델	Stage 1: Pairwise Bilinear Classifier / Stage 2~3: ATLOP Relation Head
핵심 알고리즘 / 기법	pair representation $[e_h; e_t; e_h \odot e_t]$ (Stage 1~2) 또는 refined structural vector (Stage 3) 입력. Sigmoid 기반 독립 판단, BCE loss, adaptive thresholding (TH class 추가 → score > TH인 relation만 채택). 통합 시나리오에서 ATLOP 모듈 자체는 변경 없이 입력값만 달라짐
출력	relation_logits, predicted_relations

6. Evidence Head Layer

목표	관계 예측의 근거 문장 추출 (Explainability)
사용 모델	DREEAM (Ma et al., 2023)
핵심 알고리즘 / 기법	evidence-guided attention mechanism, teacher-student self-training (gold evidence → silver evidence 생성 → student 학습), KL-divergence evidence loss, joint loss $L_{RE} + \lambda L_{evidence}$ ($\lambda=0.1$)
출력	evidence_logits, evidence sentence distribution

7. Postprocessing Layer

목표	모델 출력을 KG 저장 가능한 triple로 정제
사용 모델	Rule-based postprocessor
핵심 알고리즘 / 기법	adaptive threshold filtering (ATLOP TH 기반), duplicate removal (canonical name 기준 dedup), relation normalization (P17 → COUNTRY 등 표준 형식 통일)
출력	(head, relation, tail, score, evidence) triple 집합

8. Knowledge Graph Layer

목표	triple을 그래프 구조로 저장하고 활용 가능하게 만들
사용 모델	Neo4j (py2neo / neo4j-driver)
핵심 알고리즘 / 기법	node-edge modeling (entity → node, relation → directed edge), Cypher query, multi-edge 저장 (동일 pair에 복수 relation), multi-hop query (1~3 hop 경로 탐색), QA/RAG 연결
출력	Knowledge Graph / Relation DB 형태 결과

6. Stage 1 — Baseline RE 모델

기본적인 Document-level RE 파이프라인을 구축하여 비교 기준을 수립하는 단계이다.

6.1 Preprocessing Layer

기반 논문: Yao et al. (2019) DocRED; Wang et al. (2019) ATLOP

Step 1 — Sentence/Token 정리

문장 단위 토큰 리스트를 하나의 문서 시퀀스로 연결한다. BERT Tokenizer 적용 후 [CLS], [SEP] special token을 처리하고, 토큰별 문장 위치를 추적한다 (sentence offset mapping).

Step 2 — Entity Mention Alignment

DocRED의 sent_id + pos 정보를 문서 전체 토큰 인덱스로 변환한다. 동일 entity의 여러 mention span을 하나의 entity 구조로 묶는다. mention span 오정렬 시 entity representation 자체가 틀어지므로 핵심 품질 체크 포인트이다.

Step 3 — Entity Pair Generation

문서 내 모든 entity에 대해 방향성 있는 (head, tail) 쌍을 생성한다. entity N개 $\rightarrow N*(N-1)$ 개 pair 후보.

Step 4 — Relation Label 생성

DocRED labels를 pair별 multi-hot vector로 변환. 96차원 binary vector 생성, label 없는 pair는 no_relation 벡터 부여.

6.2 Document Encoder Layer

기반 논문: Devlin et al. (2019) BERT

항목	상세
모델	bert-base-uncased (HuggingFace Transformers)

항목	상세
입력 길이	최대 512 토큰
출력	token_hidden_states: $H \in \mathbb{R}^{L \times 768}$
Learning Rate	Encoder: $2e-5$ (AdamW, warmup 6%)

6.3 Entity Representation Layer

기반 논문: ATLOP (Zhou 2021); GAIN (Zeng 2020)

Stage 1에서는 **Mean Pooling**을 사용한다:

```
m_i = mean(H[start:end]) # 각 mention span의 평균
e_j = mean(m_i for i in entity_j.mentions) # mention들의 평균
```

6.4 Relation Extraction Layer

Pair Representation: $p(h,t) = [e_h ; e_t ; e_h \odot e_t]$

Stage 1은 고정 **threshold (0.5) + Sigmoid + BCE Loss**를 사용한다.

학습 설정 (Stage 1)

항목	값
Pre-training	train_distant.json (101K docs)
Fine-tuning	train_annotated.json (3,053 docs)
Batch size	4 (gradient accumulation 1)
Encoder LR	$2e-5$ (AdamW)
Classifier LR	$1e-4$
Warmup ratio	0.06
Max epochs	30 (pre-training) + 30 (fine-tuning)
Loss	BCE with class weight adjustment

7. Stage 2 — ATLOP + DREEAM 강화

Stage 1의 코드 구조를 그대로 유지하면서, **Relation Head**를 강화하는 단계이다.

7.1 변경점 ①: LogSumExp Pooling

기반 논문: ATLOP (Zhou et al., 2021) — Localized Context Pooling

Stage 1의 Mean Pooling을 LogSumExp Pooling으로 교체. mention 중 가장 강한 신호를 포착하며, smooth max approximation으로 작동한다.

```
m_i = log(sum(exp(H[start:end]))) # smooth max
```

7.2 변경점 ②: Adaptive Threshold (ATLOP)

기반 논문: ATLOP (Zhou et al., 2021)

각 entity pair에 "threshold" class (TH)를 추가, relation score가 TH score보다 높은 relation만 채택. 클래스 불균형에 고정 threshold보다 강건한 성능.

7.3 변경점 ③: DREEAM Evidence Head

기반 논문: DREEAM (Ma et al., 2023, EACL)

DREEAM은 **relation 예측 시 근거 문장을 동시에 학습**하는 방법. evidence head를 추가하여 attention을 유도하고, evidence loss를 보조 supervision 신호로 활용한다.

DREEAM 학습 파이프라인 (3 단계)

학습 단계	내용	데이터
1. Teacher 학습	Human-annotated data로 evidence 포함 모델 학습	train_annotated.json (gold evidence)
2. Silver Evidence 추론	Teacher가 distant data에 대해 token importance 추론	train_distant.json → .attns 파일
3. Student Self-	Silver evidence로 student 학습 → human data	distant + annotated 순차 학

학습 단계	내용	데이터
training	fine-tuning	습

Evidence Loss

```

L_total = L_RE + λ * L_evidence
L_RE      = Adaptive Threshold Loss (ATLOP)
L_evidence = KL-divergence(evidence_dist || predicted_attention)
λ          = 0.1 (BERT-base)

```

학습 설정 (Stage 2)

항목	값
Pre-training (Teacher)	train_annotated.json + gold evidence, 30 epochs
Silver evidence 추론	Teacher → train_distant.json → .attns
Self-training (Student)	train_distant.json + silver evidence, 30 epochs
Fine-tuning (Student)	train_annotated.json + gold evidence, 30 epochs
λ	0.1 (BERT-base)
GPU	NVIDIA V100 16GB

8. Stage 3 — Graph Reasoning (GAIN-lite)

Stage 2의 entity representation을 **Graph Neural Network**로 한 번 더 보정하여 inter-sentence 관계와 multi-hop 추론 능력을 강화하는 단계이다.

8.1 GAIN vs SSAN 비교 분석

기반: GAIN (Zeng 2020, EMNLP), SSAN (Xu 2021, AAAI)

항목	GAIN (Zeng 2020)	SSAN (Xu 2021)
그래프 구성	Mention + Entity dual graph	Entity-level structured attention
그래프 엣지	Co-ref + co-occur + 문서내 거리	Co-ref + co-occur (attention bias)
추론 방식	GCN message passing → repr 보정	Structured self-attention 수정
모듈 삽입 위치	Encoder 뒤, classifier 앞	Encoder 내부 (attention 수정)
적합성	★ 높음 — 모듈 삽입 용이	○ 보통 — encoder 수정 필요

결론: GAIN 구조가 Incremental Stacking에 더 적합. encoder 뒤에 GNN 모듈을 삽입하여 기존 코드를 건드리지 않는다.

8.2 GAIN-lite 그래프 구성

Edge 정의

Edge 유형	연결 조건	목적
Co-reference	동일 entity의 mention들 사이	Entity 내부 정보 전파
Co-occurrence	같은 문장에 등장하는 mention들	문장 내 관계 포착
Cross-sentence	인접 문장의 entity들 (window=1~2)	Inter-sentence 연결

8.3 GNN Message Passing

```
# GAIN-lite GNN forward
graph = build_entity_graph(entity_vectors, doc_info)
for layer in gnn_layers: # 2-layer GCN/GAT
```

```

entity_vectors = layer(entity_vectors, graph.adj_matrix)
entity_vectors = ReLU(entity_vectors)
# Refined entity vectors → ATLOP classifier에 그대로 전달
relation_logits, evidence_logits = relation_head(entity_vectors)

```

GNN 설정

항목	값
GNN 유형	GCN (기본) / GAT (ablation)
Layer 수	2
Hidden dim	768
Dropout	0.1
GNN LR	1e-4
학습 전략	Stage 2 checkpoint 로드 → GNN 추가 → 전체 fine-tuning

9. 통합 시나리오: 계층별 아키텍처 변화 분석

본 섹션은 기존 Baseline ~ Stage 2 아키텍처가 **통합 시나리오(이기종 그래프 및 U-Net 구조 도입)**로 전환될 때, 데이터가 각 계층(Layer)을 통과하며 어떻게 변화하는지 구체적으로 추적하고 그 이점을 분석한다.

9.1 전처리 및 인코딩 계층 (Preprocessing & Document Encoder)

기존 레이어 (Stage 1~2): preprocessing.py + encoder.py (BERT-base). 문서 전체(최대 512토큰)를 BERT에 통과시켜 각 토큰의 Hidden State(H)를 추출.

통합 시나리오의 변화 [확장 유지]: 기존 BERT 레이어는 그대로 유지한다. 단, 이기종 그래프(Heterogeneous Graph)를 구성하기 위해 전처리 단계에서 **문장 경계(Sentence Boundary)** 정보를 명시적으로 **노드화(Node-ify)**할 수 있도록 메타데이터 추출 로직이 추가된다.

9.2 엔티티 표현 계층 (Entity Representation Layer)

기존 레이어 (Stage 2): entity_repr.py (LogSumExp Pooling 적용). 여러 번 등장하는 Mention 중 가장 핵심적인 정보를 부드럽게(Smooth-max) 추출하여 Entity Vector(e_i)를 생성.

통합 시나리오의 변화 [그대로 유지, 입력값으로 전달]: Stage 2의 가장 큰 성과인 LogSumExp Pooling은 그 우수성이 증명되었으므로 **변경 없이 그대로 사용**한다. 이 계층의 산출물인 Entity Vector는 다음 '구조적 추론 계층'의 훌륭한 원시(Raw) 재료가 된다.

9.3 쌍 결합 vs 구조적 추론 계층 [가장 핵심적인 변화]

추출된 Entity Vector들을 엮어서 관계를 찾을 준비를 하는 단계이다. **통합 시나리오의 가장 파괴적인 혁신이 일어나는 계층**이다.

기존 레이어 (Stage 1~2): 단순 쌍 결합 (Pair-wise Concatenation)

구조: Head(e_h)와 Tail(e_t) 벡터를 단순히 이어 붙이고(Concat), 요소별 곱(Element-wise product)을 더해 1차원 Pair Vector를 생성. $[e_h; e_t; e_h * e_t]$

한계: 이 벡터는 오직 두 엔티티 h 와 t 의 정보만 담고 있다. 문서 어딘에 있는 또 다른 엔티티 k 가 이 둘의 관계를 설명하는 핵심 징검다리(Multi-hop) 역할을 하더라도, 1차원 벡터 안에 k 의 정보가 전혀 없다.

대체된 레이어 (통합 시나리오): 전역 구조 추론 계층 (Global Structural Layer)

구조: 단순 쌍 결합 코드를 삭제(또는 우회)하고, structural_encoder.py (DocuNet/U-Net 또는 이기종 GNN)를 새롭게 삽입한다.

U-Net 방식: N 개의 엔티티를 가로축과 세로축으로 배열하여 $N \times N$ 크기의 2D 이미지(Matrix)를 생성한다. 여기에 CNN 기반의 U-Net 레이어를 적용한다.

이기종 GNN 방식: Entity뿐만 아니라 Sentence(문장) 벡터를 노드로 만들어 **Entity-Sentence-Document의 3층 구조 그래프**를 구성하고 Message Passing을 수행한다.

역할: 문서 내에 존재하는 '모든 엔티티 쌍' 간의 상호작용을 한 번에(Holistically) 계산하여, 주변 문맥이 듬뿍 담긴 정제된(Refined) 2D 쌍 행렬 또는 노드 벡터를 반환한다.

왜 더 나은가? (Why it's better)

(A, B) 쌍을 판단할 때 (B, C) 쌍의 힌트를 훑쳐볼 수 있다. 2D 매트릭스 상에서 (A, B) 픽셀의 근처에는 (B, C) 픽셀이 존재하며, U-Net의 합성곱(Convolution) 필터가 이 주변 픽셀들의 정보를 압축해서 (A, B) 픽셀에 섬어준다.

이 수학적 연산 덕분에, 기존 모델이 짚짚매던 **다단계 간접 추론(Multi-hop Reasoning)**이 자연스럽게 해결된다. "A는 B에 위치하고, B는 C에 위치하므로, A는 C에 위치한다"는 논리가 레이어 내부 연산에 녹아들게 된다.

9.4 관계 분류 및 증거 추출 계층 (Relation & Evidence Classification)

기존 레이어 (Stage 2): relation_head.py (Adaptive Thresholding + DREEAM Evidence Head). 국소적인(Local) Pair Vector를 입력받아 각 클래스별 점수를 매기고, 가변 임계값(TH)을 기준으로 최종 관계를 출력. 동시에 어느 문장에 Attention이 쏠리는지 계산(Evidence).

통합 시나리오의 변화: 모듈 자체(ATLOP 논리와 DREEAM 손실 함수)는 변경하지 않는다. 하지만 **입력받는 값이 완전히 달라진다.** (Raw Vector → Refined Structural Vector). 문서 전체의 구조를 파악하고 온(Refined) 2D 쌍 행렬이나 이기종 그래프 노드를 바탕으로 ATLOP 분류와 Evidence 추출을 수행한다.

10. Knowledge Graph Layer (Neo4j)

10.1 Postprocessing

Adaptive Threshold 적용: ATLOP의 학습된 TH로 relation 채택 여부 결정

Duplicate 제거: Entity canonical name 기준 deduplication

Relation Normalization: 모델 출력 label → KG 표준 형식 (P17 → COUNTRY)

10.2 Neo4j 그래프 구성

Entity canonical name을 node로, relation triple을 directed edge로 저장. Multi-edge 허용.

```
// Node 생성
MERGE (e:Entity {name: $name}) SET e.type = $type, e.aliases = $aliases
// Edge 생성
MATCH (h:Entity {name: $head}), (t:Entity {name: $tail})
CREATE (h)-[:REL {type: $rel, score: $score, evidence: $evi}]->(t)
// Multi-hop 쿼리
MATCH p=(a)-[*1..3]->(b) WHERE a.name='Steve Jobs' RETURN p
```

10.3 응용 연결

QA 시스템: 그래프 쿼리 결과를 LLM 컨텍스트로 활용

Graph-based RAG: 관련 entity-relation 서브그래프를 retrieval context로 주입

추천 시스템: Entity 연결 패턴 기반 유사 entity 추천

11. 구현 계획 및 코드 구조

11.1 레이어별 기반 논문 및 참고 코드

레이어	기반 논문	참고 코드 / 구현 포인트
Preprocessing	DocRED (Yao 2019)	ATLOP GitHub prepro.py 기반
Encoder	BERT (Devlin 2019)	HuggingFace bert-base-uncased
Entity Repr.	ATLOP (Zhou 2021)	LogSumExp pooling (atlop GitHub)
Structural Reasoning	GAIN (Zeng 2020)	GCN/GAT message passing (gain GitHub)
Relation Head	ATLOP + DREEAM (Ma 2023)	Adaptive threshold + evidence head
Knowledge Graph	Neo4j	py2neo / neo4j-driver

11.2 프로젝트 디렉토리 구조

```

docre-kg/
├── data/
│   ├── docred/           # DocRED raw data
│   └── meta/             # rel2id.json, rel_info.json
├── src/
│   ├── preprocessing.py  # DocRED → feature 변환
│   ├── encoder.py        # BERT encoder wrapper
│   ├── entity_repr.py    # mention/entity pooling (mean → logsumexp)
│   ├── structural_encoder.py # 이기종 GNN / U-Net (Stage 3)
│   ├── relation_head.py  # ATLOP + DREEAM classifier
│   ├── model.py          # 전체 모델 forward (stage flag로 분기)
│   ├── postprocessing.py  # threshold, dedup, normalization
│   └── kg_builder.py      # Neo4j 연동
├── scripts/
│   ├── train.py / evaluate.py / infer_evidence.py / build_kg.py
├── configs/
│   ├── stage1.yaml / stage2.yaml / stage3.yaml
└── README.md

```

11.3 model.py Forward 분기 설계

```

class DocREModel(nn.Module):
    def __init__(self, config):
        self.encoder = BertModel.from_pretrained('bert-base-uncased')
        self.entity_repr = EntityRepr(config)          # mean or logsumexp

```



```
self.structural_encoder = None          # Stage 3에서만 활성화
self.relation_head = RelationHead(config) # fixed or ATLOP+DREEAM

def forward(self, batch, stage='stage1'):
    H = self.encoder(batch.input_ids, batch.attention_mask)
    entity_vecs = self.entity_repr(H, batch.entity_spans)

    if stage == 'stage3' and self.structural_encoder:
        # 이기종 GNN 또는 U-Net으로 entity vector 보정
        graph = build_hetero_graph(entity_vecs, batch.doc_info)
        entity_vecs = self.structural_encoder(entity_vecs, graph)

    logits = self.relation_head(entity_vecs, batch.entity_pairs)
    return logits # {relation_logits, evidence_logits (if stage2+)}
```

12. 팀원 역할 분담

담당	담당 레이어	핵심 구현 내용
전처리 담당	Preprocessing Layer	DocRED 파싱, entity mention alignment, pair 생성, label 벡터화
모델 담당	Encoder + Entity Repr. + Relation Head	BERT 인코딩, mention pooling, ATLOP/DREEAM 분류기
후처리 + 그래프 담당	Structural Encoder + KG + Postprocessing	GNN/U-Net reasoning, Neo4j 구성, multi-hop 쿼리

13. 핵심 참고 논문

1. Yao et al. (2019) "DocRED: A Large-Scale Document-Level Relation Extraction Dataset." ACL 2019.
2. Zhou et al. (2021) "ATLOP: Document-Level RE with Adaptive Thresholding and Localized Context Pooling." AAAI 2021.
3. Ma et al. (2023) "DREEAM: Guiding Attention with Evidence for Improving Document-Level RE." EACL 2023.
4. Zeng et al. (2020) "GAIN: Document-Level RE with Graph Inference Network." EMNLP 2020.
5. Xu et al. (2021) "SSAN: Entity Structure Within and Throughout." AAAI 2021.
6. Zhang et al. (2021) "DocuNet: Document-Level RE with U-shaped Network." ACL 2021.
7. Devlin et al. (2019) "BERT: Pre-training of Deep Bidirectional Transformers." NAACL 2019.
8. Liu et al. (2019) "RoBERTa: A Robustly Optimized BERT Pretraining Approach." arXiv 2019.
9. Tan et al. (2022) "KD-DocRE: Document-Level RE with Knowledge Distillation." ACL Findings 2022.
10. Kipf & Welling (2017) "Semi-Supervised Classification with GCN." ICLR 2017.
11. Veličković et al. (2018) "Graph Attention Networks." ICLR 2018.

참고 GitHub 리포지토리:

GAIN: <https://github.com/PKUnlp-icler/GAIN>

SSAN: <https://github.com/BenfengXu/SSAN>

DREEAM: <https://github.com/YoumiMa/dreeam>

ATLOP: <https://github.com/wzhouad/ATLOP>

DocRED: <https://huggingface.co/datasets/thunlp/docred>